Part I, Week 01, Day 04 — SVD & PCA Foundations

0) কেন SVD দরকার (Eigendecomposition-এর সীমাবদ্ধতা)

Day 03-এ আপনি Eigendecomposition শিখেছেন। এটি শক্তিশালী, কিন্তু একটি বড় সীমাবদ্ধতা আছে:

1. Eigendecomposition সাধারণত শুধু square matrix (n x n) এর জন্য সরাসরি কাজ করে।
2. সব square matrix-এ যথেষ্ট eigenvector নাও থাকতে পারে (matrix defective হতে পারে), তখন decomposition "ভালোভাবে" হয় না।

কিন্তু বাস্তব ডেটা সাধারণত rectangular হয়:

- m samples (ডেটা পয়েন্ট/রো)
- n features (কলাম)

অর্থাৎ A এর shape হয় m x n, যেখানে m != n প্রায়ই।

এই বাস্তব সমস্যার জন্য SVD হলো "সব কাজের কাজী" (Swiss Army Knife) কারণ:

- যে কোনো real matrix A (m x n), square হোক বা rectangular, SVD দিয়ে decomposition সম্ভব।
- decomposition টা orthogonal basis ব্যবহার করে, তাই numerically stable এবং ML কাজে খুব সুবিধাজনক।

1) SVD থিওরেম (মূল সূত্র)

যে কোনো real matrix A (m x n) কে লেখা যায়:

A = U * Sigma * V^T

এখানে:

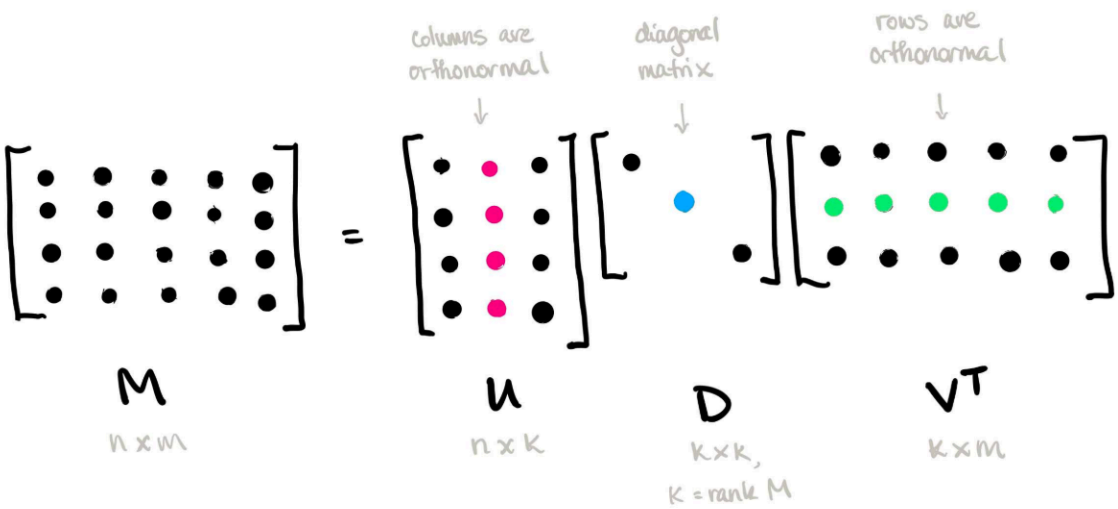
- A: m x n
- U: m x m
- Sigma: m x n
- V^T: n x n

সবচেয়ে গুরুত্বপূর্ণ ব্যাপার:
U এবং V দুটোই orthogonal matrix (real case)।

Orthogonal মানে:

- U^T * U = I (m x m identity)
- V^T * V = I (n x n identity)

এটার মানে কী?
U এবং V এর columns গুলো একে অপরের সাথে perpendicular (orthonormal) basis vector।



2) তিনটা ম্যাট্রিক্সের "অ্যানাটমি"

2.1) U — Left Singular Vectors (m x m)

U এর columns গুলোকে বলা হয় left singular vectors।

Interpretation (intuition):

- U column space / output space এর জন্য orthonormal basis দেয়।
- ML context এ: U অনেকটা "samples/observations" এর মধ্যে সম্পর্ক/structure capture করে।

সহজ কথা:

- আপনি যখন A কে V-space থেকে rotate করে U-space এ নিয়ে যান, তখন U আপনাকে "row-wise pattern / sample structure" বুঝতে সাহায্য করে

2.2) Sigma — Singular Values (m x n diagonal)

Sigma হলো diagonal matrix (rectangular diagonal).

Diagonal মানে:

- Sigma এর মূল diagonal এ কিছু সংখ্যা থাকবে: sigma_1, sigma_2, ..., sigma_r

- অন্য সব জায়গায় 0

এখানে:

- sigma_i >= 0
- sorted: sigma_1 >= sigma_2 >= ... >= 0

Interpretation:

- প্রতিটি sigma_i বলে দেয় corresponding component কতটা “strong/important”।
- বড় sigma => ওই direction/component এ data variation/energy বেশি।

ML context:

- PCA তে আমরা বড় singular values গুলো রাখি, ছোটগুলো drop করি => dimensionality reduction।

2.3) V^T — Right Singular Vectors (n x n)

V^T এর rows (অথবা V এর columns) হলো right singular vectors।

Interpretation:

- V features-space (input space) এর orthonormal basis দেয়।
- ML context এ: V এর vectors হলো feature combination / principal directions।
- PCA language এ: V এর প্রথম কয়েকটা column = principal components directions (feature space directions)

সহজ কথা:

- V বলে দেয়: “features গুলো মিশে কোন কোন direction এ সবচেয়ে meaningful variation তৈরি করছে।”

3) SVD কে “rotate + scale + rotate” হিসেবে চিন্তা করুন

A = U * Sigma * V^T কে আপনি এভাবে ভাবতে পারেন:

Step 1: V^T দিয়ে rotate/reflect (feature space এ)
Step 2: Sigma দিয়ে scale (প্রতিটি axis আলাদা পরিমাণে stretch/shrink)
Step 3: U দিয়ে আবার rotate/reflect (sample space/output space এ)

অর্থাৎ A আসলে একটি linear transformation, যা:

- প্রথমে coordinate system বদলায় (V^T)
- তারপর magnitude বদলায় (Sigma)
- তারপর নতুন coordinate system এ বসায় (U)

4) কেন SVD PCA-এর foundation

PCA মূলত চায়:

- এমন directions খুঁজতে যেখানে data variance সবচেয়ে বেশি
- variance বেশি মানে তথ্য বেশি (প্রায়ই)

SVD এ:

- V এর columns দেয় principal directions (feature combinations)
- Sigma এর sigma_i দেয় প্রতিটি direction কতটা গুরুত্বপূর্ণ

তাই PCA করতে সাধারণত:

- Top-k singular values (sigma_1..sigma_k) রাখা হয়
- Corresponding V columns রাখা হয়
- এভাবে data কে lower dimension এ project করা হয়

5) Quick shape check (Doc-safe)

ধরি A হলো m x n

Then:

- U: m x m
- Sigma: m x n
- V^T: n x n

Multiplication consistent:
U (m x m) * Sigma (m x n) = (m x n)
(m x n) * V^T (n x n) = (m x n) = A

2) Geometric Intuition: “Rotate, Scale, Rotate” (SVD এর জ্যামিতিক ধারণা)

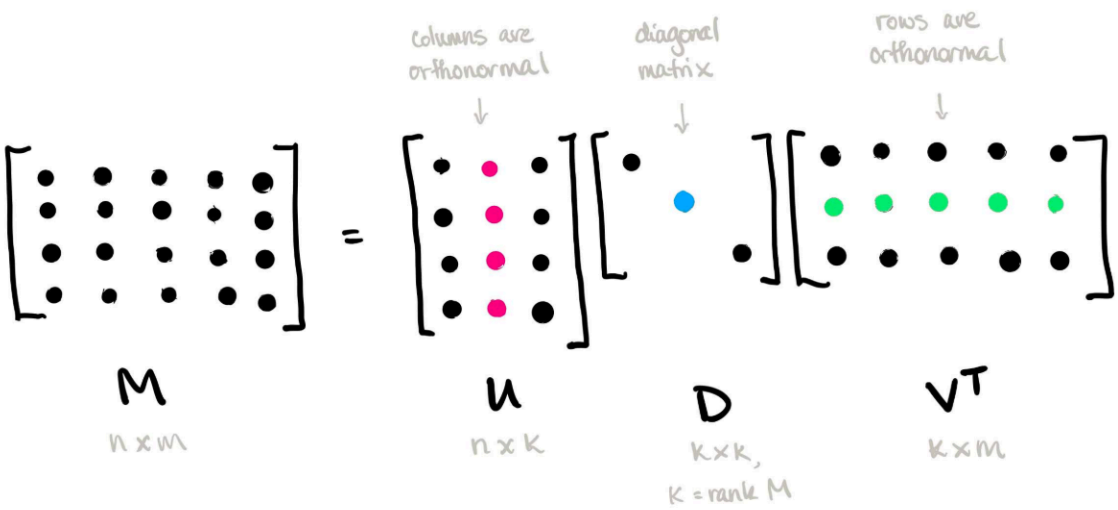
SVD বলে:

A = U * Sigma * V^T

এটা শুধু একটা algebraic ভাঙা-গড়া না—এর খুব পরিষ্কার geometric মানে আছে।
যে কোনো linear transformation A: R^n -> R^m (মানে n-dimensional input কে m-dimensional output এ ম্যাপ করে) — এটাকে সবসময় ৩টা ধারাবাহিক ধাপে ভাঙা যায়:

1. V^T : Rotate/Reflect (input space এ)
2. Sigma : Scale (stretch/squish)
3. U : Rotate/Reflect (output space এ)

সোজা কথা:
প্রথমে ঘুরাও, তারপর টানো/চাপাও, তারপর আবার ঘুরাও।



আগে “Rotate/Reflect” আর “Scale” মানে কী?

Rotate (Rotation)

- vector এর length বদলায় না
- শুধু direction বদলায়
- angle preserve করে

Reflect (Reflection)

- mirror-এর মতো উল্টে দেয় (একটা axis বরাবর flip)
- length বদলায় না

Rotate/Reflect একসাথে বলা হচ্ছে কারণ orthogonal matrix (যেমন U, V) এই দুইটাই করতে পারে।

Orthogonal matrix property :

- Q^T * Q = I
  এখানে Q হতে পারে U বা V.

Scale (Scaling)

- vector এর কিছু component বড় হয়, কিছু ছোট হয়
- length সাধারণত বদলায়

Step 1) V^T (Rotate/Reflect): “input space কে ঠিক axis-এ align করা”

আপনার original input vector ধরা যাক x (n x 1)।

প্রথমে apply হয়:
x' = V^T * x

এটার মানে:

- input space এ coordinate system বদলাচ্ছে
- আপনি যেন ডেটাকে ঘুরিয়ে এমনভাবে বসাচ্ছেন যাতে “true axes of variance” এর সাথে align হয়

Intuition:

- আপনার features space এ variance (বা তথ্য) সবসময় x-axis/y-axis এর মতো “সোজা” অক্ষ বরাবর থাকে না
- অনেক সময় variance একটা তির্যক (diagonal) দিক বরাবর বেশি
- V^T সেই তির্যক দিকগুলোকে নতুন coordinate axis বানিয়ে দেয়

PCA link:

- V এর columns হলো principal directions (feature combinations)
- V^T মানে “principal coordinate system” এ projection/rotation

Step 2) Sigma (Scale): “নতুন axis বরাবর stretch/squish করা”

এরপর apply হয়:
z = Sigma * x'

Sigma হলো diagonal-ish (rectangular diagonal) matrix, যার diagonal এ singular values থাকে:
Sigma diagonal entries: sigma_1, sigma_2, ..., sigma_r
শর্ত:

- sigma_1 >= sigma_2 >= ... >= 0

Scaling intuition:

- প্রতিটি axis বরাবর scale factor আলাদা:
    - 1st axis scale হবে sigma_1 দিয়ে
    - 2nd axis scale হবে sigma_2 দিয়ে
    - ...
- sigma বড় হলে ওই direction এ vector বেশি টানা হবে (stretch)
- sigma ছোট হলে ওই direction এ কম টানা হবে বা চাপা হবে (squish)
- sigma = 0 হলে ওই direction “চাপা পড়ে যায়” (information collapses)

Machine Learning intuition:

- বড় sigma মানে ওই component বেশি গুরুত্বপূর্ণ (data energy/variance বেশি)
- PCA তে তাই top-k sigma রাখলে বেশি তথ্য preserve হয়

Step 3) U (Rotate/Reflect): “output space এ final orientation বসানো”

শেষে apply হয়:
y = U * z

এটার মানে:

- scaled vector কে output space (R^m) এ নিয়ে গিয়ে final orientation সেট করা
- আবারও length preserve করে শুধু direction/placement বদলাতে পারে (rotation/reflect)

ইন্টুইশন:

- Step 2 শেষে আপনার vector “ঠিক অক্ষ বরাবর scale” হয়েছে
- কিন্তু এখনও সেটা output space এর final coordinate frame এ বসেনি
- U সেটাই করে: output space এ কোন দিকে/কোন plane এ যাবে সেটা নির্ধারণ করে

পুরোটা একসাথে: “Why Rotate-Scale-Rotate works for any A”

কোনো linear transformation A সাধারণত একসাথে:

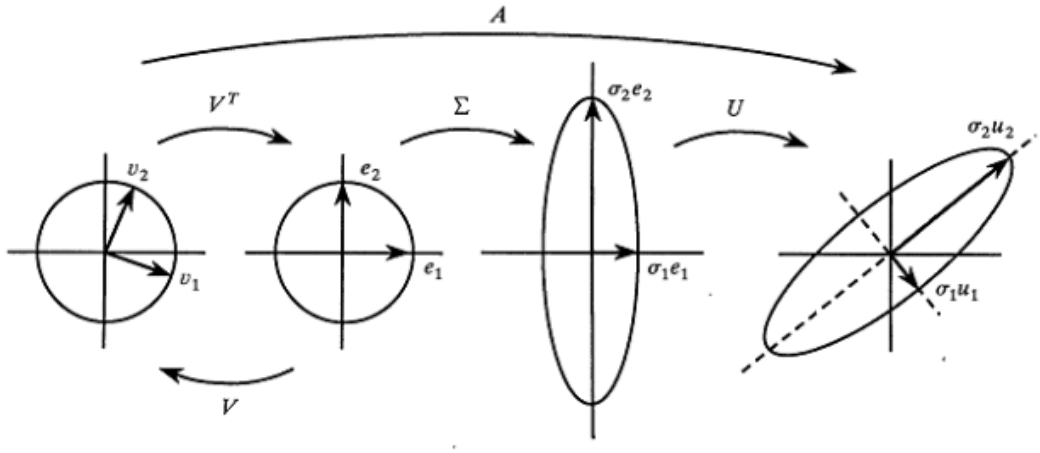
- ঘোরায়
- টানে/চাপে
- dimension mix করে

SVD বলে:
এই জটিল কাজটাকে সবসময় এভাবে ভাঙা যায়:

1. V^T দিয়ে input কে এমন basis এ নাও যেখানে কাজটা “সহজ” (axis-aligned) হয়ে যায়
2. Sigma দিয়ে axis ধরে ধরে শুধু stretch/squish করো (সবচেয়ে সহজ linear transform)
3. U দিয়ে result কে output space এ ঠিক orientation এ বসাও

ছোট্ট mental picture

ধরুন input space এ একটি unit circle (2D case)।



A transform করলে:

- circle সাধারণত ellipse হয়ে যায় (টানা/চাপা + ঘোরানো)

SVD interpretation:

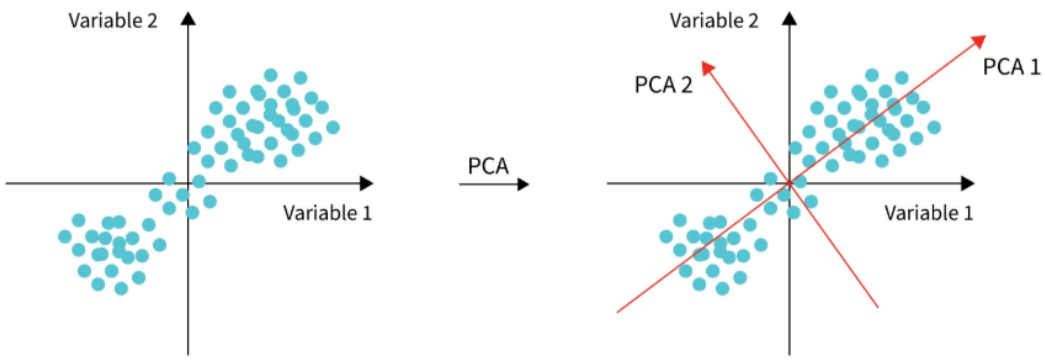
- V^T: circle কে আগে ঘুরিয়ে “ellipse axes” এর সাথে align করলো
- Sigma: circle কে টেনে ellipse বানালো (major/minor axes lengths = sigma values)
- U: ellipse কে final অবস্থানে ঘুরিয়ে বসালো

3) The Bridge to PCA — SVD দিয়ে PCA কেন "ফ্রি" পাওয়া যায়

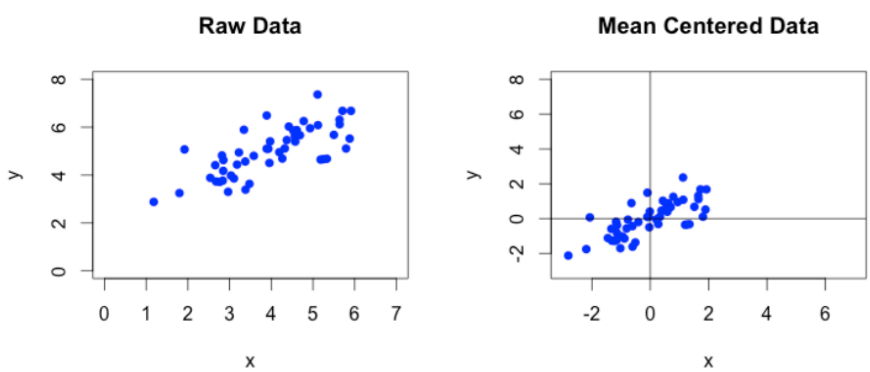
1) PCA আসলে কী করতে চায়

PCA (Principal Component Analysis) চায়:

- ডেটার এমন কিছু direction খুঁজতে, যেদিকে variance (ছড়ানো/তথ্য) সবচেয়ে বেশি
- সেই direction গুলোকে principal components বলা হয়
- এরপর top-k direction রেখে dimension কমানো হয় (dimensionality reduction)



2) PCA-র ক্লাসিক পথ: covariance matrix + eigenvectors



ধরি আপনার mean-centered data matrix হলো X (m x n):

- m = number of samples

- n = number of features
- mean-centered মানে: প্রতিটি feature column থেকে তার mean বাদ দেওয়া হয়েছে (so column mean = 0)

PCA সাধারণত covariance matrix বানায়:

C = (1/(m-1)) * X^T * X

Shapes:

- X: (m x n)
- X^T: (n x m)
- X^T * X: (n x n) -> square matrix
- C: (n x n)

এরপর PCA করে:

- C এর eigenvectors বের করে
- top eigenvectors = principal components directions

3) সমস্যাটা কোথায়: X^T * X কেন "unstable"

X^T * X গণনা করলে:

- বড় ডেটায় numerical issues বাড়ে
- condition number আরও খারাপ হতে পারে (ইনভার্স/ইজেন বের করা কঠিন)
- floating-point rounding error amplify হতে পারে

এই কারণে বাস্তবে "C = X^T X" approach অনেক সময় numerically shaky।

4) Stable পথ: X এর সরাসরি SVD

SVD দিয়ে X কে ভাঙি:

X = U * Sigma * V^T

Shapes:

- U: (m x m)
- Sigma: (m x n)
- V^T: (n x n)

Key idea:

- আমরা covariance বের করতে X^T X না বানিয়ে, আগে X এর SVD বের করি

5) Covariance equation এ SVD বসানো

Covariance (constant factor বাদ দিয়ে) বলা যায়:

$C \propto X^T * X$

এখন $X = U * Sigma * V^T$ বসাই:

$C \propto (U * Sigma * V^T)^T * (U * Sigma * V^T)$

Transpose নিয়ম:

- $(ABC)^T = C^T * B^T * A^T$

সুতরাং:

$(U * Sigma * V^T)^T = V * Sigma^T * U^T$

এখন গুণ করি:

$C \propto (V * Sigma^T * U^T) * (U * Sigma * V^T)$

Bracket করে লিখলে:

$C \propto V * Sigma^T * (U^T * U) * Sigma * V^T$

এখন U orthogonal, তাই:

$U^T * U = I$

সুতরাং:

$C \propto V * Sigma^T * Sigma * V^T$

এখন $Sigma^T * Sigma$ আসলে diagonal matrix, যার diagonal এ থাকবে:

$sigma_1^2, sigma_2^2, ..., sigma_r^2$

তাই:

$C \propto V * (Sigma^2) * V^T$

(এখানে “$Sigma^2$” বলতে বোঝানো হচ্ছে $Sigma^T Sigma$ থেকে পাওয়া diagonal entries squared)

6) এটা eigendecomposition এর মতোই

Eigendecomposition form:

C = Q * Lambda * Q^T

এখানে:

- Q = eigenvectors matrix
- Lambda = diagonal eigenvalues

আমাদের derived form:

C ∝ V * (Sigma^T Sigma) * V^T

Match করলে দেখা যায়:

- Q corresponds to V
- Lambda corresponds to (Sigma^T Sigma) scaled by (1/(m-1))

অর্থাৎ:

V এর columns (right singular vectors) = PCA principal component directions

এটাই আপনার লেখার মূল conclusion।

7) Eigenvalues আর singular values এর সম্পর্ক

Original covariance:

C = (1/(m-1)) * X^T * X

আমরা পেয়েছি:

X^T * X = V * (Sigma^T Sigma) * V^T

তাই:

C = (1/(m-1)) * V * (Sigma^T Sigma) * V^T

এখান থেকে প্রতিটি eigenvalue:

lambda_i = (sigma_i^2) / (m-1)

Doc-safe summary:

- principal component directions = columns of V
- eigenvalues = singular values squared divided by (m-1)

8) “SVD gives PCA for free” — মানে কী

PCA করতে আপনার দরকার:

1. principal directions
2. explained variance (eigenvalues)

SVD থেকে আপনি সরাসরি পান:

- directions: V
- strengths: sigma_i (আর variance = sigma_i^2/(m-1))

অতএব covariance matrix explicitly বানানো ছাড়াই PCA করা যায়।

9) Practical ML benefit: কেন SVD বেশি stable

- X^T X বানালে errors amplify হতে পারে
- SVD algorithms (যেমন LAPACK ভিত্তিক) সাধারণত বেশি numerically stable
- বড়/ill-conditioned data তে PCA করতে SVD route বেশি reliable

4) Low-Rank Approximation: Data Compression (Truncated SVD)

1) মূল আইডিয়া: সব তথ্য সমান গুরুত্বপূর্ণ না

SVD এ আমরা লিখি:

A = U * Sigma * V^T

এখানে Sigma এর diagonal এ singular values থাকে:

sigma_1 >= sigma_2 >= ... >= sigma_r >= 0

এই sorting (বড় থেকে ছোট) খুব গুরুত্বপূর্ণ। কারণ সাধারণত:

- বড় singular values = data এর “signal/structure” বেশি capture করে
- ছোট singular values = বেশিরভাগ ক্ষেত্রে “noise/irrelevant detail” এর মতো আচরণ করে

অর্থাৎ, Sigma এর শুরু দিকের কয়েকটা মান (top few) রেখে দিলে অনেক সময় data এর প্রধান pattern ধরে রাখা যায়।

2) Truncated SVD কী

Truncated SVD মানে হলো:

- শুধু top-k singular values রাখবো
- বাকি singular values গুলোকে 0 করে দেব

k হচ্ছে একটা ছোট সংখ্যা, যেমন:

- k << r
- r = rank(A) বা max possible rank

3) Low-rank approximation formula (doc-safe)

Top-k রেখে তৈরি করা approximation কে লেখা হয়:

A ≈ A_k = U_k * Sigma_k * V_k^T

এখানে matrix sizes:

- A: m x n

- U_k: m x k (U এর প্রথম kটা column)
- Sigma_k: k x k (top-k singular values এর diagonal matrix)
- V_k^T: k x n (V^T এর প্রথম kটা row)
  (equivalently, V_k: n x k)

Multiplication check:
(U_k: m x k) * (Sigma_k: k x k) = m x k
(m x k) * (V_k^T: k x n) = m x n = A_k

4) Low-rank কেন "compression"

একটা full matrix A store করতে লাগে:

- m*n সংখ্যক value

কিন্তু A_k store করতে লাগে:

- U_k: m*k values
- Sigma_k: k values (diagonal only) বা k*k (যদি full store করেন)
- V_k^T: k*n values

মোট প্রায়:
m*k* + *k* + *k*n (বা m*k* + *k*k + k*n)

যদি k ছোট হয়, তখন:
m*k* + *k*n << m*n

ফলে memory কম লাগে এবং computationও অনেক দ্রুত হয়।

5) "Signal vs Noise" intuition (কেন ছোট singular value বাদ দিতে পারি)

Singular values (sigma_i) effectively বলে দেয়:

- ith component data কে explain করতে কতটা শক্তিশালী

যদি sigma_i খুব ছোট:

- সেই direction এ data variation খুব কম
- অনেক সময় সেটা measurement noise, background detail, বা weak pattern

তাই sigma_i ছোট হলে সেটাকে drop করলে:

- noise কমে
- data এর major structure থাকে

6) Engineering applications

6.1) Image compression

একটা grayscale image কে matrix ধরা যায়:

- rows = pixel rows
- cols = pixel cols
- entry = intensity

Truncated SVD করলে:

- বড় structure (edges, big shapes) থাকে
- tiny details/noise কমে
- কম storage এ image “প্রায় একই” দেখায়

6.2) NLP: Latent Semantic Analysis (LSA)

NLP তে term-document matrix হয়:

- rows = words/terms
- cols = documents
- entry = frequency/weight (TF-IDF etc.)

SVD করে top-k রাখলে:

- শব্দ ও ডকুমেন্টকে কম dimensional “latent topics” এ map করা যায়
- synonymy/polysemy handle করতে সাহায্য করে
- search/retrieval উন্নত হয়

6.3) Recommendation systems: Matrix factorization (Netflix-style)

Recommender এ সাধারণত ratings matrix থাকে:

- rows = users
- cols = items/movies
- entry = rating (অনেকগুলো missing)

Low-rank assumption:

- মানুষের পছন্দ কয়েকটা latent factor দিয়ে explain করা যায় (যেমন action vs romance, comedy taste, etc.)

Truncated SVD / matrix factorization ধারণা থেকে:

- user embeddings (U_k side)

- item embeddings (V_k side)
- strength/scales (Sigma_k)

Week 12 এ আপনি যে Netflix-like system বানাবেন, সেটার mathematical foundation এখানেই।

7) Rank বলতে কী বোঝাচ্ছে (quick intuition)

Rank roughly মানে:

- matrix এ কয়টা independent direction/component আছে

Low-rank approximation মানে:

- আসল matrix এর বদলে কম component দিয়ে একটা “best possible” approximation রাখা

Practical viewpoint:

- rank কম মানে: ডেটা আসলে কয়েকটা underlying factor দিয়ে গঠিত